Lab – Working with Docker Images

To Begin with the Lab

Summary

In this lab, we learn how to manage Docker images using different Docker commands. We explore how to list available images, inspect image details in JSON format, understand how Docker uses locally stored images to create containers, and remove images from the local system using docker rmi.

We also learn why Docker does not allow an image to be deleted while containers are referencing it, even if those containers are stopped. Finally, we remove the containers, delete the image, and run the container again to understand how Docker automatically downloads an image from Docker Hub when it is not available locally.

Lab Steps

Step 1: Open Ubuntu Terminal

• Open Docker Desktop.

• Open Ubuntu terminal (WSL) on Windows

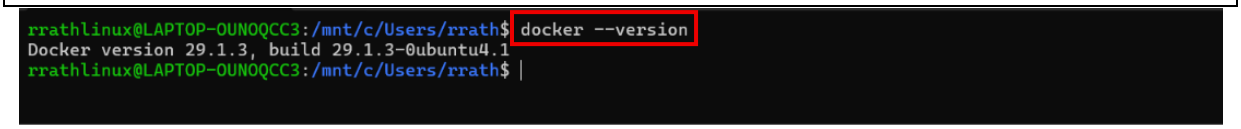
• Install docker Ubuntu.

```
sudo apt update
sudo apt install docker.io -y
sudo systemctl start docker
sudo systemctl enable docker
```

• Ensure Docker Desktop is running.

• Verify Docker installation:

```
docker –version
```



Step 2: Run Nginx Container in Foreground

• Execute the following command:

```
docker run nginx
```

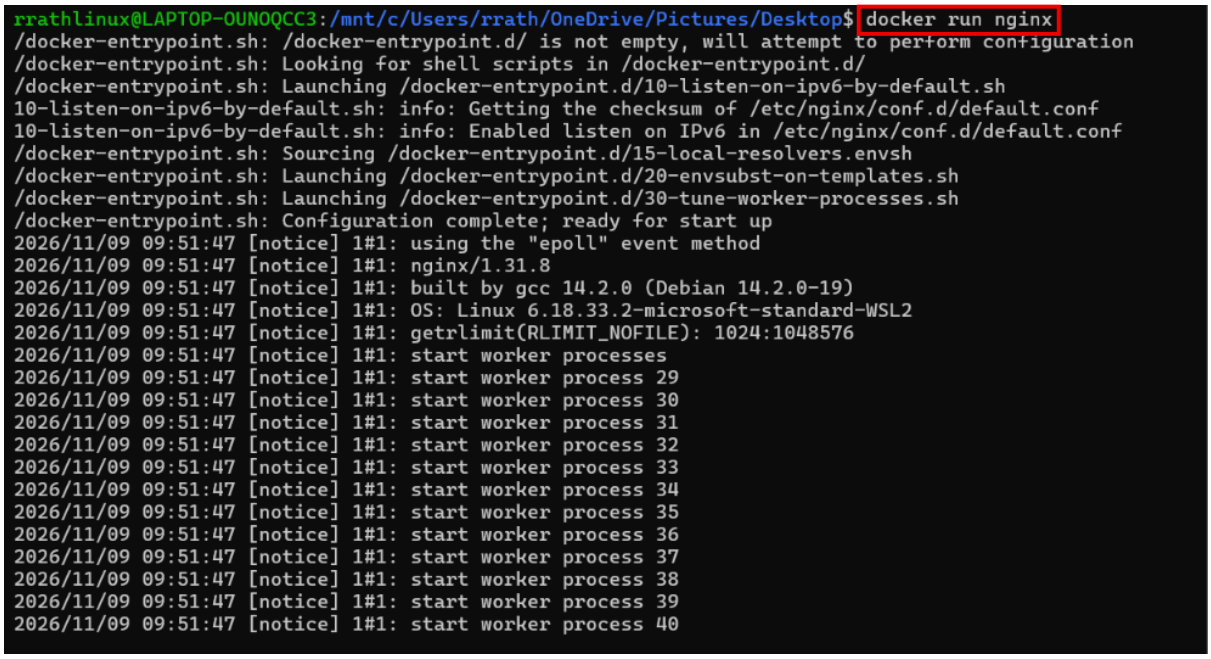
• Docker creates a new container from the Nginx image.

• Nginx starts running inside the container.

• Container logs appear continuously in the terminal.

Observation:

The terminal becomes occupied because the container is running in foreground mode.



Step 3: List Available Docker Images

• Execute:

```
docker images
```

• This command displays all Docker images available locally.

Output information:

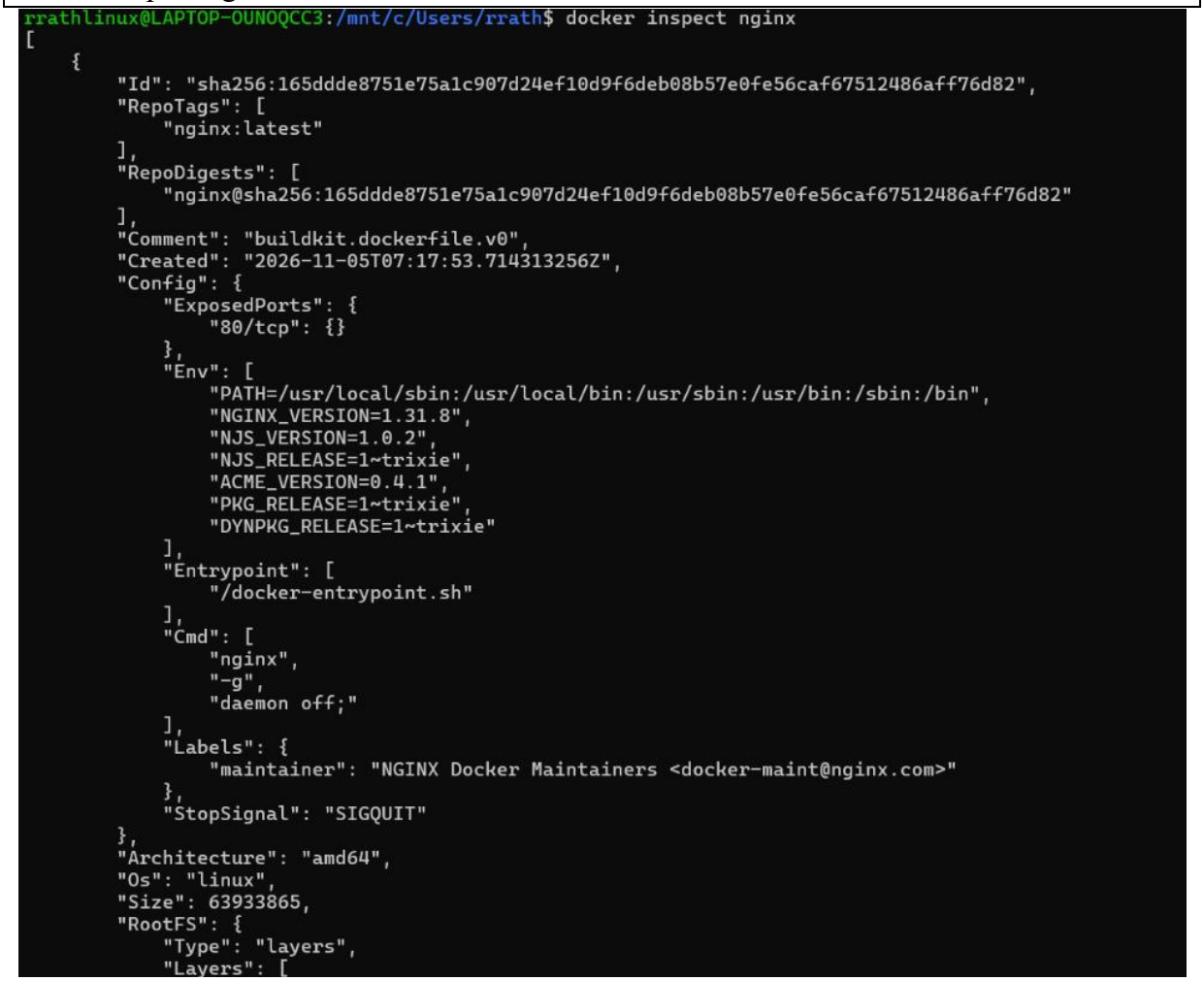
Column	Description
REPOSITORY	Image name.
TAG	Image version.
IMAGE ID	Unique image identifier.
CREATED	Image creation time.
SIZE	Image size.



Step 4: Inspect Docker Image

• Execute:

docker inspect nginx



• Docker displays detailed information about the Nginx image.

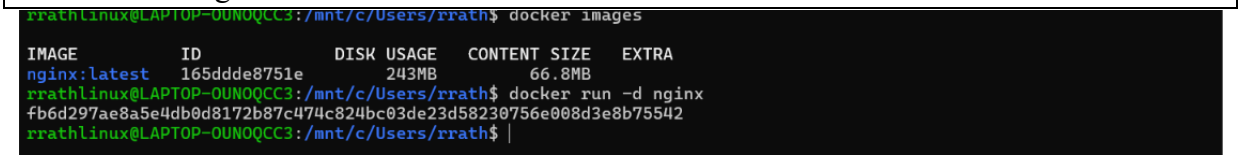
• The output is displayed in JSON format.

• Examine the image ID, architecture, operating system, configuration, and other details.

Step 5: Run a Container Using the Local Image

• Execute:

docker run -d nginx



• Docker checks whether the Nginx image exists locally.

• Since the image is already available, Docker does not download it again.

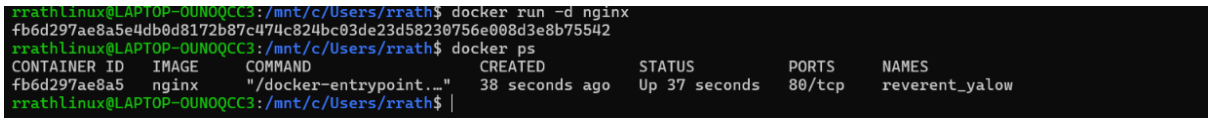
• Docker creates a new container using the existing image.

• Verify the running container:

```
docker ps
```

Observation:

Docker directly uses the locally available Nginx image to create the container.



Step 6: Try to Remove the Nginx Image

• Execute:

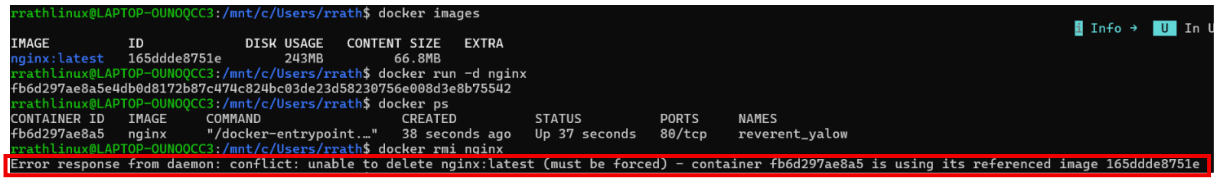
```
docker rmi nginx
```

• Docker may display an error:

Error response from daemon:

conflict: unable to remove nginx

(image is being used by stopped container)



Why does this happen?

Docker does not allow an image to be removed while existing containers are referencing it.

Even if containers are stopped, they can still reference the image.

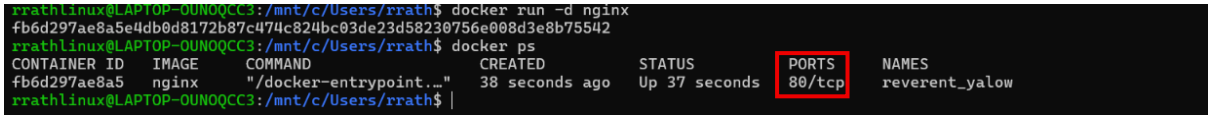
Step 7: Check Running Containers

• Execute:

```
docker ps
```

• This command displays currently running containers.

• Identify the containers created using the Nginx image.



Step 8: Stop Running Containers

• Copy the container ID.

• Execute:

```
docker stop <container-id>
```

Now the containers should no longer appear in the running containers list.

Important: The containers still exist in the system, but their status is now Exited.

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker rmi nginx
Error response from daemon: conflict: unable to delete nginx:latest (must be forced) - container fb6d297ae8a5 is using its referenced image 165ddde8751e
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker stop fb6d297ae8a5
fb6d297ae8a5
```

Step 9: Display All Containers

• Execute:

```
docker ps -a
```

• This command displays all containers, including running and stopped containers.

• Identify the containers that are using the Nginx image.

Step 10: Remove Containers

• To delete a specific container, execute:

```
docker rm <container-id>
```

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker stop fb6d297ae8a5
fb6d297ae8a5
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker rm fb6d297ae8a5
fb6d297ae8a5
```

Step 11: Remove the Nginx Image

• Execute:

```
docker rmi nginx
```

• Docker removes the Nginx image from the local machine.

• Docker may display information about deleted image layers.

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker stop fb6d297ae8a5
fb6d297ae8a5
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker rm fb6d297ae8a5
fb6d297ae8a5
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker rmi nginx
Untagged: nginx:latest
Deleted: sha256:165ddde8751e75a1c907d24ef10d9f6deb08b57e0fe56caf67512486aff76d82
```

• Verify image removal:

```
docker images
```

If the image was successfully deleted, it will no longer appear in the list.

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker images

IMAGE   ID             DISK USAGE   CONTENT SIZE   EXTRA
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$
```

Step 12: Run Nginx Again

Now the Nginx image is no longer available locally.

• Execute:

```
docker run -d -p 8080:80 nginx
```

Explanation:

Command	Description
docker run	Creates and starts a container.
-d	Runs the container in the background.
-p	Publishes a container port to the host.
8080	Host machine port.
80	Nginx container port.
Nginx	Docker image name.

• Docker checks whether the Nginx image exists locally.

• Since the image was deleted, Docker downloads it again from Docker Hub.

• After downloading, Docker creates and starts a new container.

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker run -d -p 8080:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
6d2e43921e9e: Pull complete
1724c05f72c6: Pull complete
37fb86c2a767: Pull complete
512ac5e7ce8e: Pull complete
bbb080c71d50: Pull complete
1962e1ff75b4: Pull complete
7cb4701ce7cd: Pull complete
4582dace95e2: Download complete
d2c7e31f04a3: Download complete
Digest: sha256:165ddde8751e75a1c907d24ef10d9f6deb08b57e0fe56caf67512486aff76d82
Status: Downloaded newer image for nginx:latest
034104ca8138000ca41790317251928ada1cd65033166d9bdc66b46c323bae9a
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$
```

Step 13: Verify the Container

• Execute:

```
docker ps
```

• Verify that the Nginx container is running.

```
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker run -d -p 8080:80 nginx
Unable to find image 'nginx:latest' locally
latest: Pulling from library/nginx
6d2e43921e9e: Pull complete
1724c05f72c6: Pull complete
37fb86c2a767: Pull complete
512ac5e7ce8e: Pull complete
bbb080c71d50: Pull complete
1962e1ff75b4: Pull complete
7cb4701ce7cd: Pull complete
4582dace95e2: Download complete
d2c7e31f04a3: Download complete
Digest: sha256:165ddde8751e75a1c907d24ef10d9f6deb08b57e0fe56caf67512486aff76d82
Status: Downloaded newer image for nginx:latest
034104ca8138000ca41790317251928ada1cd65033166d9bdc66b46c323bae9a
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$ docker ps
CONTAINER ID   IMAGE     COMMAND                  CREATED          STATUS          PORTS                                   NAMES
034104ca8138   nginx     "/docker-entrypoint.…"   45 seconds ago   Up 44 seconds   0.0.0.0:8080->80/tcp, [::]:8080->80/tcp   mystifying_germain
rrathlinux@LAPTOP-OUNOQCC3:/mnt/c/Users/rrath$
```

• Open your browser and enter:

http://localhost:8080

• You should see the default Nginx welcome page.

• You can also open Docker Desktop and navigate to Containers to see the running container.

